

The story behind Graph Websearch Agent

How a single LLM call turned into a five-agent state machine that argues with itself until the answer is good enough.

A multi-agent web research system built on LangGraph
github.com/yashnagaria/Graph-Websearch-Agent

1. The problem that started it

Ask a language model "what is the population of Tokyo right now?" and you get a confident number. It is often wrong, and it is always stale — the model is answering from training data that froze months or years ago. The obvious fix is to let it search the web. So you bolt on a search API, paste the top ten snippets into the prompt, and ask again.

That works, right up until it doesn't:

- **Snippets lie by omission.** A search snippet is 160 characters chosen by a ranking algorithm, not by relevance to *your* question. The model confidently summarises a fragment.
- **One shot, no recovery.** If the search term was bad, the whole answer is bad, and nothing in the pipeline notices.
- **No self-awareness.** The model has no mechanism to say "this source didn't actually answer the question, let me look somewhere else."

The classic RAG chain is a straight line: `query → search → stuff context → answer`. A straight line cannot go back. And *going back* is exactly what research is.

So the question became: **what if the pipeline could loop?**

2. Why a graph and not a chain

The moment you want retries, you are no longer describing a chain — you are describing a **state machine**. Nodes do work. Edges decide who works next. One of those edges has to be conditional, or nothing can ever loop.

That is precisely what [LangGraph](#) models, so the architecture fell out of the problem statement:

Node	Job	Output
planner	Read the question, decide what to actually search for	JSON: <code>search_term</code> , <code>overall_strategy</code>
search	Hit the search engine	A formatted SERP
selector	Read the whole SERP and pick <i>one</i> URL worth reading	JSON: <code>selected_page_url</code> + why
scraper	Fetch that page, strip it to text	<code>{source, content}</code>
reporter	Write an answer grounded in that page	Markdown
reviewer	Critique the draft against the original question	JSON: structured feedback
router	Decide: ship it, or send someone back to work	JSON: <code>next_agent</code>

The whole design hinges on one edge. In `agent_graph/graph.py` :

```
graph.add_conditional_edges(
    "router",
    lambda state: pass_review(state=state),
)
```

Every other edge is fixed. This one reads the router's JSON and returns a node name — which may be `planner` (the search term was wrong), `selector` (wrong page), `reporter` (right source, bad write-up), or `final_report` (we're done). That single line is what separates "a RAG script" from "a research loop."

3. The design decisions, and why

Separate agents instead of one big prompt

It is tempting to write one giant prompt: *"search, pick a source, write an answer, check it."* It fails, for a reason that is easy to miss: a model that writes an answer and then grades its own answer in the same context window is grading its own homework with the answer sheet open. It almost always says "looks good."

Splitting reporter and reviewer into **separate LLM calls with different system prompts and no shared reasoning** means the reviewer sees the draft as an artefact, not as its own work. The critiques get noticeably sharper.

One page at a time, not ten

The selector deliberately picks **a single URL**. This felt wrong at first — why throw away nine results? But it makes the system honest. Every claim in the final report traces to one named source you can click. If that source is bad, the reviewer says so and the loop picks another. Ten sources blended into one answer gives you a report nobody can audit.

Force JSON everywhere it matters

Four of the five agents return JSON against a declared schema in `prompts/prompts.py`. The router in particular *must* return `{"next_agent": "..."} — a conditional edge that receives prose instead of a node name crashes the graph. Gemini's response_mime_type: application/json and OpenAI's json_object mode do the heavy lifting; a code-fence stripper catches the rest.`

A shared state, not passed messages

Every node reads and writes one `AgentGraphState` (`states/state.py`). Each field is annotated with LangGraph's `add_messages` reducer, so writes **append** rather than overwrite:

```
planner_response: Annotated[list, add_messages]
```

This is what makes the loop intelligent rather than amnesiac. When the router sends work back to the planner, the planner receives the reviewer's feedback *and* can see what it already tried. It does not propose the same failed search term twice.

4. The architecture in detail

4.1 The layers

The system is six layers deep, and the important property is that **no layer above knows which model provider sits underneath it**. Every model client, whether it is talking to Gemini over REST or to a local Ollama daemon, returns the same `HumanMessage` object.

PRESENTATION	streamlit_app.py app/chat.py (Chainlit) all three consume workflow.stream(...)	app/app.py (CLI)
ORCHESTRATION	agent_graph/graph.py StateGraph: 9 nodes, 7 fixed edges, 1 conditional edge, 1 entry + 1 finish point	
AGENTS	agents/agents.py Agent base class + 7 subclasses	
CONTRACTS	prompts/prompts.py system prompts + JSON output schemas	states/state.py the shared state schema and its accessors
CAPABILITY	models/*.py 6 provider clients	tools/*.py search + scraper
CONFIG	utils/helper_functions.py	config/config.yaml

Swapping Gemini for Groq changes one dropdown value. Nothing in the graph, the agents or the prompts is aware it happened.

4.2 The state object

Everything flows through one `TypedDict` in `states/state.py`. There is no message passing between agents — they communicate only by reading and writing this shared object.

Field	Type	Written by	Read by
research_question	str	the caller, once	every agent
planner_response	append-only list	planner	search tool
serper_response	append-only list	search tool	selector
selector_response	append-only list	selector	scraper tool
scraper_response	append-only list	scraper tool	reporter
reporter_response	append-only list	reporter	reviewer, final report
reviewer_response	append-only list	reviewer	router, planner, selector, reporter
router_response	append-only list	router	the conditional edge

Field	Type	Written by	Read by
<code>final_reports</code>	append-only list	final report node	the caller
<code>end_chain</code>	append-only list	end node	nothing — it is a terminator

Every field except the question carries LangGraph's `add_messages` reducer:

```
planner_response: Annotated[list, add_messages]
```

Append, never overwrite. This is the single most important decision in the data model. When the router sends work back to the planner on the third iteration, the planner can see its own two previous attempts *and* every critique the reviewer has written. That is what stops it proposing the same failed search term twice — the loop has memory, so it is iterative rather than merely repetitive.

There is a subtlety worth knowing if you read the agent code. Every agent ends with:

```
def update_state(self, key, value):
    self.state = {**self.state, key: value}  # returns the ENTIRE state
```

So each node hands the whole state back to LangGraph, not just the field it changed — which looks like it should duplicate every message on every step. It does not, because `add_messages` assigns a UUID to each message and merges by that id: messages already in the state are recognised and replaced in place, while the one genuinely new value (a raw `str`) is converted to a `HumanMessage` with a fresh id and appended. Only the changed field grows.

`get_agent_graph_state(state, key)` is the read side, and it is deliberately dumb: `"..._all"` returns the whole history, `"..._latest"` returns `[-1]`. Agents that need context ask for `_all`; agents that need the current artefact ask for `_latest`.

4.3 Node contracts

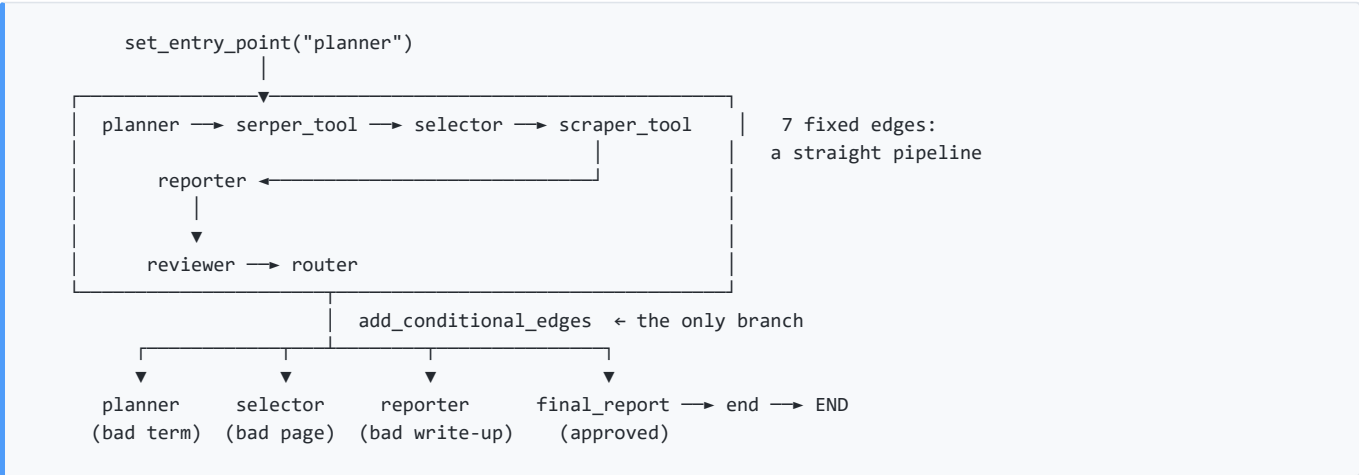
Nine nodes. Seven call an LLM or a tool; two are bookkeeping.

Node	Reads	Produces	LLM mode
<code>planner</code>	question, <code>reviewer_latest</code>	<code>{search_term, overall_strategy, additional_information}</code>	JSON
<code>serper_tool</code>	<code>planner_latest</code>	formatted SERP text	none — HTTP
<code>selector</code>	question, SERP, <code>selector_all</code> , <code>reviewer_latest</code>	<code>{selected_page_url, description, reason_for_selection}</code>	JSON
<code>scraper_tool</code>	<code>selector_latest</code>	<code>{source, content}</code> capped at 4000 chars	none — HTTP
<code>reporter</code>	question, scraped page, <code>reporter_all</code> , <code>reviewer_latest</code>	Markdown report	free text

Node	Reads	Produces	LLM mode
reviewer	question, reporter_latest , reviewer_all , full state	structured critique	JSON
router	question, reviewer_all	{next_agent}	JSON
final_report	reporter_latest	promotes the draft to final_reports	none
end	—	writes end_chain , terminates	none

The reporter is the **only** agent that runs in free-text mode (`self.get_llm(json_model=False)`). Everything else is forced through `response_mime_type: application/json` with a declared schema in `prompts/prompts.py` . That asymmetry is deliberate: prose is the product, and every other output is a control signal that something downstream has to parse.

4.4 Control flow



Every edge except one is hardcoded. The loop exists entirely because of:

```
graph.add_conditional_edges("router", lambda state: pass_review(state=state))
```

`pass_review` reads the last `router_response` , parses its JSON, and returns a node name as a string. LangGraph resolves that string to the next node. Return `"planner"` and the whole pipeline re-runs with a new search term; return `"reporter"` and only the write-up is redone against the page already in state.

Three ways a run terminates, which matters because an agent loop that cannot stop is a billing incident:

- Success** — the router returns `final_report` , which promotes the draft and flows to `end` .
- Recursion limit** — `{"recursion_limit": 40}` caps total node executions. LangGraph raises once it is exceeded.
- The fail-safe** — anything `pass_review` cannot understand (unparsable JSON, a missing `next_agent` , an unknown node name) returns `"end"` . This is what makes a transient API error cost you one run instead of an exception, and it is a change I made only after watching a `503` throw a `KeyError` out of the conditional edge and discard twelve completed steps.

4.5 The provider abstraction

`Agent.get_llm()` is the whole of it — a dispatch table from a provider string to a client pair:

```
def get_llm(self, json_model=True):
    if self.server == 'gemini':
        return GeminiJSONModel(...) if json_model else GeminiModel(...)
    if self.server == 'openai':
        return get_open_ai_json(...) if json_model else get_open_ai(...)
    # groq, claude, ollama, vllm follow the same shape
```

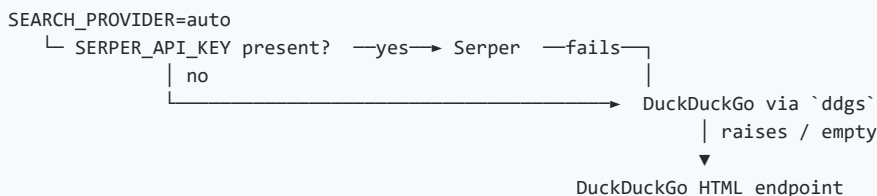
Every provider ships exactly two classes — a JSON one and a text one — and both expose a single method, `invoke(messages) -> HumanMessage`. `messages` is always a two-element list, a system turn and a user turn.

The contract is not just the signature. **A client never raises.** Every model client catches its own exceptions and returns `HumanMessage(content='{"error": "..."}')`. That is why a dead API degrades into a reviewer politely critiquing an error string rather than a stack trace, and it is why the Streamlit layer has to check for `{"error": ...}` payloads explicitly before treating any text as a report.

Only the OpenAI client is a LangChain wrapper. The other five are ~60 lines of `requests` each, which is why the repo has no provider SDKs to keep in sync.

4.6 The tool layer

Search resolves a provider at call time rather than at import, so a key pasted into the sidebar mid-session takes effect immediately. It degrades down a ladder:

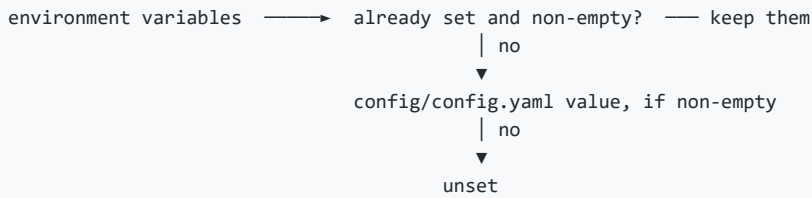


Each rung needs no configuration from the one above it, so the system has no hard dependency on any paid service.

The scraper fetches exactly one URL with a browser user agent and a 20-second timeout, flattens the HTML with BeautifulSoup's `stripped_strings`, rejects the result if more than 30% of characters are non-ASCII (a cheap and surprisingly effective binary-garbage detector), and truncates to 4000 characters. A 403 or a timeout is written into state as scraper *output*, not raised — so the reviewer sees an empty source and the router reroutes.

4.7 Configuration resolution

One function, `load_config`, called at the top of every model and tool module:



Environment always wins. On Streamlit Cloud the keys arrive from `st.secrets`, get pushed into `os.environ` before the graph is built, and the empty committed YAML never touches them. Getting this precedence backwards is what silently broke the first cloud deploy.

4.8 Execution and streaming

`workflow.stream(inputs, {"recursion_limit": n})` is a generator yielding one dict per completed node, shaped `{node_name: state_update}`. The Streamlit layer renders each as it arrives, which is why the UI shows the agents working rather than a spinner.

Two things about that stream are easy to get wrong. It yields the node's **return value**, not the reduced state — so the field a node just wrote is still a raw `str`, while every other field holds the `HumanMessage` list it was handed. And because the agents are synchronous, the run occupies the Streamlit script thread for its whole duration; the Chainlit UI wraps it in `cl.make_async` for the same reason.

4.9 The failure model

The design assumption is that **every external call fails sometimes**, so each failure has a defined destination rather than a traceback.

Failure	Where it is caught	What happens
LLM 503 / 429 / network	model client	4 retries with 2s/5s/12s backoff
LLM still failing after retries	model client	returns <code>{"error": ...}</code> ; the run continues
LLM returns fenced or array JSON	<code>GeminiJSONModel</code>	fence stripped, first dict extracted
LLM returns no text at all	<code>_extract_text</code>	raises with <code>finishReason</code> named
Serper down or out of quota	<code>run_search</code>	falls through to DuckDuckGo
<code>ddgs</code> backend times out	<code>search_duckduckgo</code>	falls through to the HTML endpoint
Page returns 403 / times out	<code>scrape_website</code>	written to state as content; router reroutes
Page is binary garbage	<code>is_garbled</code>	replaced with an error string; router reroutes
Router emits unparsable JSON	<code>pass_review</code>	graph ends cleanly, last draft preserved
Loop will not converge	LangGraph	recursion limit stops it
Anything else	<code>streamlit_app.py</code>	error plus traceback rendered in the page

4.10 What a single question costs

A clean run with no retries is **7 LLM calls** — planner, selector, reporter, reviewer, router, then final report — plus one search request and one page fetch. Each reviewer rejection adds three more calls (reporter, reviewer, router). The observed run in §8 took 145 seconds and 14 node executions across two reroutes.

The expensive input is the scraped page at 4000 characters; the reviewer is second, because it is handed the entire state object in its prompt. Both are worth knowing before raising the recursion limit much above 40.

5. The parts that fought back

An honest build log is more useful than a highlight reel. These are the things that actually broke.

The config loader ate its own keys. The original `load_config` looped over `config.yaml` and wrote every value into the environment — including empty ones, which it replaced with placeholder strings like `'default_openai_api_key'`. On a laptop with a filled-in YAML, fine. On Streamlit Cloud, where keys arrive through `st.secrets` and the YAML is empty, it silently overwrote every real key with garbage a split second before the first API call. The fix inverts the precedence: **an environment variable that already has a value always wins**, and an empty YAML entry never overwrites anything.

Gemini 2.5 Flash answered with silence. Switching to a 2.5 model produced responses with a `candidates` array, a valid `finishReason`, and *no text*. The model was spending its entire output budget on internal reasoning tokens before it ever started writing. The fix is one line in `models/gemini_models.py` — `"thinkingConfig": {"thinkingBudget": 0}` — plus error handling that reports `finishReason` instead of throwing an opaque `KeyError`, because "no text (finishReason: MAX_TOKENS)" is a debuggable message and `KeyError: 'parts'` is not.

Half the web returns 403 to Python. The scraper called `requests.get(url)` with the default `python-requests/2.x` user agent. A large share of news sites and docs pages reject that outright. Sending a browser user agent and a timeout fixed most of it — and crucially, a 403 is *handled*, not fatal: it goes into the state as scraper output, the reviewer sees an empty source, and the router sends the selector back to pick a different page. **The failure becomes an input to the loop**, which is the whole point of the architecture.

Search needed a credit card. The original build required a Serper key. Anyone who cloned the repo hit a `KeyError: 'SERPER_API_KEY'` on the first run. Now `tools/google_serper.py` resolves a provider at runtime: use Serper if a key exists, fall back to DuckDuckGo (no signup, no key) if it does not — and fall back again if a Serper call fails mid-run. **One free Gemini key is now enough to run the entire system.**

Two of the three documented model ids were dead. The code defaulted to `gemini-2.0-flash`. Against a freshly issued AI Studio key it returns *404: no longer available to new users*. So did `gemini-2.5-flash`. Worse, both still appear in the `ListModels` response — the catalogue endpoint lists models the key cannot actually call, so you only find out by sending a real request. The fix was to probe every candidate with a live call, default to a verified id, and replace the free-text model box with a dropdown of ids that were actually confirmed to work.

Free-tier Gemini fails a lot, and each failure cost a whole agent turn. Under load the API returns `503 high demand` frequently. Because every agent swallows its own errors into an `{"error": ...}` payload, a single 503 propagated: the reporter "wrote" an error string, the reviewer earnestly critiqued that error string, and the router burned a loop rerouting because of it. Retrying four times with backoff *inside* the model client is far cheaper than letting the graph absorb the failure — and the UI now collects those error payloads separately so one can never be rendered as the final report.

A hardcoded path to somebody's Google Drive. `app/chat.py` pointed at `G:/My Drive/Data-Centric Solutions/...`. It ran on exactly one machine on Earth. Now it resolves relative to the file.

6. From terminal to a URL anyone can open

The original interface was `input()` in a terminal with `termcolor` output. That is a fine way to debug an agent and a terrible way to show one to anybody else.

The Streamlit rewrite (`streamlit_app.py`) exists because of one insight: **the intermediate steps are the product.** A research agent that prints an answer after ninety seconds of silence feels broken and untrustworthy. The same agent that shows you " 🗨️ *Planner chose the search term 'Tokyo population 2025'* → 🎯 *Selector picked the Statistics Bureau page* → ⚖️ *Reviewer says the figure is stale, try again*" feels like watching a colleague work.

So the app consumes `workflow.stream(...)` and renders each node's update as it arrives — the planner's strategy, the raw SERP, the chosen URL, the scraped text, each draft, each critique, each routing decision. The final report lands at the bottom with a download button. Failures render as a visible error with a traceback in an expander, rather than a blank page.

Deployment shaped the code too: the entry point moved to the repo root, Chainlit moved out to `requirements-chainlit.txt` so the Cloud build stays small, and every key became overridable from the sidebar so the app is usable by someone who has never seen the repo.

7. What it is good for

Use case A — Time-sensitive factual lookup

When the answer changed after the model's training cutoff. Prices, populations, standings, release notes, "who currently holds X." A plain LLM answers confidently and wrongly; this loop goes and reads a page from this week, and the reviewer specifically checks the answer's recency.

Use case B — Cited research briefs

When "the model said so" is not good enough. Every report names the URL it was written from, because the reporter only ever sees one scraped page. That makes the output checkable — you can open the source and disagree with it, which you cannot do with a model's recollection.

Use case C — Question triage on unfamiliar topics

When you do not know the right search term yet. The planner's job is to translate a vague human question into a good query, and the loop rewrites that query when the results disappoint. It is a better first pass than a human guessing keywords cold.

Use case D — A teaching harness for agent design

When you want to see multi-agent orchestration without a framework hiding it. Every prompt is in one file, every model client is ~60 lines of `requests`, and the graph is twenty lines of edges. You can break one agent and watch exactly how the system compensates.

Where it is the wrong tool

Not for questions the model already knows (slow and expensive for no benefit), not for questions needing twenty sources synthesised (it reads one page per pass by design), and not for anything behind a login or a hard paywall.

8. A worked example, end to end

This is a **real run**, not an idealised one — the trace below is what the graph actually did on `gemini-3.6-flash` with DuckDuckGo search and no Serper key. It took 145 seconds and 14 steps.

Question: "What are the headline features of Python 3.13?"

[1] 🧠 **Planner.** Rewrote the casual question into something a release-notes page would match:

```
{
  "search_term": "Python 3.13 headline features whats new",
  "overall_strategy": "Search for official Python 3.13 release notes and technical summaries
                      to identify the major headline features..."
}
```

[2] 🔍 **Search.** DuckDuckGo, no API key, 4558 characters of results.

[3] 🎯 **Selector.** Out of ten results it chose the primary source over the blog posts:

```
{
  "selected_page_url": "https://docs.python.org/3/whatsnew/3.13.html",
  "description": "Official Python documentation outlining all major new features..."
}
```

[4] 📄 **Scraper.** Fetched it with a browser user agent and stripped it to text.

[5] 🚫 **Reporter — failed.** Gemini returned `503 high demand`. The agent swallowed it into an `{"error": ...}` payload rather than throwing.

[6] ⚖️ **Reviewer.** Read that payload and — correctly — refused it:

```
"The reporter response failed due to an API execution error. Please regenerate the report providing a summary of
Python 3.13 headline features..."
```

[7] 🔄 **Router.** `{"next_agent": "reporter"}` — **the conditional edge fired and sent work backwards.** This is the whole thesis of the architecture, triggered by a real failure rather than a contrived one.

[8] 🏆 **Reporter — succeeded.** With the source still in state, it produced:

- **Improved Interactive Interpreter** — a greatly improved REPL [1]
- **Experimental Free-Threaded CPython** — runs with the GIL disabled (PEP 703) [1]
- **Experimental JIT Compiler** (PEP 744) [1]
- **Improved Error Messages** — tracebacks coloured by default [1]
- **Defined Semantics for** `locals()` (PEP 667) [1]
- **Type Parameter Defaults**, mobile platform support, and removal of the "dead batteries" (PEP 594) [1]

Sources: [1] <https://docs.python.org/3/whatsnew/3.13.html>

Every claim carries a citation, because the reporter only ever saw that one page.

[9–11] The reviewer hit a 503 of its own, the router again routed back to the reporter, and the reporter reproduced an equivalent report.

[12] 🏛️ **Reviewer — approved.**

"The report accurately and comprehensively summarizes the headline features of Python 3.13..."

[13] 🔄 **Router — failed, and the graph survived it.** Another 503. Before this was fixed, `review_data["next_agent"]` raised a `KeyError` that killed the entire run and discarded twelve steps of completed work. Now an unreadable routing decision ends the graph cleanly and the caller keeps the last good draft.

[14] 🏁 **End.** The approved report is returned and rendered with a download button, above the full expandable trace.

What this run demonstrates: the loop rerouted twice, recovered from three separate upstream API failures, never crashed, and still produced a correct, cited answer. A straight-line RAG chain hits the 503 at step 5 and returns an error to the user.

9. What I would build next

- **Parallel selectors.** Read the top three sources concurrently and let the reporter cross-check them. LangGraph supports fan-out; the state reducers are already append-only, so the shape is right — the reporter prompt is what needs rethinking.
 - **A budget, not just a recursion limit.** Cap the run by tokens and wall-clock, not only by step count, and surface the cost in the UI.
 - **Persistent checkpoints.** LangGraph's `SqliteSaver` would let a run be paused, inspected and resumed — and would turn the trace into something you can replay.
 - **Reviewer scoring.** Have the reviewer emit a numeric confidence alongside its prose, so the router can stop on "good enough" instead of "the reviewer ran out of objections."
-

10. The one-paragraph version

I built a web research agent as a LangGraph state machine rather than a RAG chain, because research needs to be able to go back. Five agents — planner, selector, reporter, reviewer, router — share an append-only state; the router owns the only conditional edge, and it can send work back to any earlier agent when the reviewer rejects a draft. Keeping the reviewer in a separate LLM call from the reporter is what makes the critiques real. Along the way I fixed a config loader that overwrote live API keys with placeholders, a Gemini 2.5 model that spent its whole output budget thinking, and a scraper that half the internet rejected for its user agent — and I added a keyless DuckDuckGo fallback so the whole thing runs on one free API key. The Streamlit front-end streams every agent hand-off live, because with an agent, the reasoning is the product.